

Assignment Implementation Guide

This document provides detailed guidance for implementing the Audio Customer Support Agent assignment.

Important: Library Restrictions

PROHIBITED LIBRARIES - Use of these will result in automatic disqualification:

- **Pipecat** - High-level conversational AI framework
- **LiveKit** - Real-time audio/video infrastructure
- **Daily** - Video calling infrastructure
- **Agora** - Real-time engagement platform
- **Twilio Voice** - Telephony API with built-in speech processing
- **AssemblyAI Universal** - All-in-one speech AI platform
- **Deepgram Aura** - Text-to-speech with built-in conversation handling
- **OpenAI Realtime API** - Real-time conversational API
- **Any other high-level conversational AI platforms or frameworks**

ALLOWED LIBRARIES:

- Individual component libraries (OpenAI Whisper, ElevenLabs, etc.)
- Direct API calls to speech services
- Core Python libraries (asyncio, requests, etc.)
- Framework libraries (FastAPI, Streamlit, LangChain)
- Database and vector store libraries (ChromaDB, etc.)

The goal is to implement the pipeline architecture yourself, not use pre-built conversational AI solutions.

Assignment Overview

You'll build a complete audio-based customer support agent using the STT → LLM → TTS pipeline. The project provides a blueprint with abstract classes and TODO sections that you need to complete.

Architecture Flow

Audio Input → STT → LLM Agent (with RAG) → TTS → Audio Output

Implementation Strategy

Recommended Implementation Order

1. **LLM Agent with RAG** (Start here - easiest to test)
2. **STT Component** (Choose your preferred service/model)
3. **TTS Component** (Choose your preferred service/model)
4. **Pipeline Integration** (Connect all components)
5. **Testing & Validation** (Use provided UI and test utilities)

Phase 1: LLM Agent with RAG Implementation

File: `src/llm/agent.py`

What's Already Done

- **16 comprehensive customer support documents** covering returns, shipping, support, warranty, payments, etc.
- **Automatic ChromaDB setup** with document ingestion and embeddings
- **Complete agent infrastructure** with LangChain integration
- **Knowledge base persistence** (survives server restarts)

Your Task: Implement RAG Search

You only need to complete **ONE method**: `_rag_search()` in `CustomerSupportAgent` class.

```

async def _rag_search(self, query: str) -> str:
    """
    TODO: Implement ChromaDB query to find relevant documents.

    Available resources:
    - self.collection: ChromaDB collection with embedded documents
    - 16 predefined customer support documents already ingested

    Implementation steps:
    1. Query ChromaDB for relevant documents
    2. Format results for readability
    3. Return formatted response
    """

```

Step-by-Step Implementation

Step 1: Query ChromaDB

```

results = self.collection.query(
    query_texts=[query],
    n_results=3,
    include=['documents', 'metadatas', 'distances']
)

```

Step 2: Format Results

```

formatted_results = []
for doc, meta in zip(results['documents'][0], results['metadatas'][0]):
    formatted_results.append(f"***{meta['title']}**\n{doc}")
return "\n\n".join(formatted_results)

```

Testing Your RAG Implementation

```

# Use the provided test utility
python src/utils/kb_test.py

# This will show you:
# - Knowledge base structure
# - Sample test queries
# - Your implementation results

```

See `docs/RAG_IMPLEMENTATION_GUIDE.md` for detailed RAG implementation instructions.

Phase 2: STT Implementation

File: `src/stt/base_stt.py`

Service Options

API-Based Services (Recommended for beginners):

- **Deepgram** (Free \$200 credits): `pip install deepgram-sdk`
- **AssemblyAI** (Free tier): `pip install assemblyai`
- **Azure Speech**: `pip install azure-cognitiveservices-speech`

Local Models (No API keys needed):

- **OpenAI Whisper** (Highly recommended): `pip install openai-whisper`
- **Wav2Vec2**: `pip install transformers torch`
- **Vosk**: `pip install vosk`

Implementation Tasks

Complete these methods in `STTService` class:

1. Initialize Method

```
async def initialize(self) -> None:
    # Example for Deepgram:
    from deepgram import DeepgramClient
    api_key = self.config.get("api_key")
    self.client = DeepgramClient(api_key)
    self.is_initialized = True

    # Example for Whisper (local):
    import whisper
    model_name = self.config.get("model", "base")
    self.client = whisper.load_model(model_name)
    self.is_initialized = True
```

2. Transcribe Method

```

async def transcribe(self, audio_bytes: bytes, **kwargs) -> str:
    # Example for Deepgram:
    response = await self.client.listen.prerecorded.v("1").transcribe_file(
        {"buffer": audio_bytes},
        {"model": "nova-2", "smart_format": True}
    )
    return response["results"]["channels"][0]["alternatives"][0]["transcript"]

# Example for Whisper (local):
import tempfile
with tempfile.NamedTemporaryFile(suffix=".wav") as temp_file:
    temp_file.write(audio_bytes)
    result = self.client.transcribe(temp_file.name)
    return result["text"]

```

Testing STT

```

# Test via API endpoint (after implementing)
curl -X POST http://localhost:8000/debug/stt -F "audio=@test_audio.wav"

```

Phase 3: TTS Implementation

File: `src/tts/base_tts.py`

Service Options

API-Based Services:

- **ElevenLabs** (Free 10k chars/month): `pip install elevenlabs`
- **OpenAI TTS** (High quality): Included in `openai` package
- **Azure Speech**: `pip install azure-cognitiveservices-speech`

Local Models (No API keys needed):

- **Edge TTS** (Free Microsoft voices): `pip install edge-tts`
- **Coqui TTS** (Various models): `pip install TTS`
- **Parler TTS**: `pip install parler-tts`

Implementation Tasks

Complete these methods in `TTSService` class:

1. Initialize Method

```
async def initialize(self) -> None:
    # Example for ElevenLabs:
    from elevenlabs import ElevenLabs
    api_key = self.config.get("api_key")
    self.client = ElevenLabs(api_key=api_key)
    self.voice_id = self.config.get("voice_id", "21m00Tcm4TlvDq8ikWAM")
    self.is_initialized = True

    # Example for Edge TTS (local):
    # No initialization needed, just set flag
    self.voice = self.config.get("voice", "en-US-AriaNeural")
    self.is_initialized = True
```

2. Synthesize Method

```
async def synthesize(self, text: str, **kwargs) -> bytes:
    # Example for ElevenLabs:
    audio_stream = self.client.text_to_speech.stream(
        text=text,
        voice_id=self.voice_id,
        model="eleven_turbo_v2_5"
    )
    audio_bytes = b""
    for chunk in audio_stream:
        audio_bytes += chunk
    return audio_bytes

    # Example for Edge TTS (local):
    import edge_tts
    communicate = edge_tts.Communicate(text, self.voice)
    audio_bytes = b""
    async for chunk in communicate.stream():
        if chunk["type"] == "audio":
            audio_bytes += chunk["data"]
    return audio_bytes
```

Testing TTS

```
# Test via API endpoint (after implementing)
curl "http://localhost:8000/chat/audio/Hello%20world" --output test_output.mp3
```

Phase 4: Pipeline Integration

File: `src/pipeline.py`

Implementation Tasks

1. Initialize Pipeline

```
async def initialize(self) -> None:
    # Initialize STT
    self.stt = STTService(self.config.stt_config)
    await self.stt.initialize()

    # Initialize LLM Agent
    self.llm_agent = CustomerSupportAgent(self.config.llm_config)
    await self.llm_agent.initialize()

    # Initialize TTS
    self.tts = TTSService(self.config.tts_config)
    await self.tts.initialize()

    self.is_initialized = True
```

2. Process Audio Pipeline

```
async def process_audio(self, audio_bytes: bytes, **kwargs) -> bytes:
    # Step 1: STT
    text_input = await self.stt.transcribe(audio_bytes)

    # Step 2: LLM
    agent_response = await self.llm_agent.process_query(text_input)

    # Step 3: TTS
    response_audio = await self.tts.synthesize(agent_response)

    return response_audio
```

Phase 5: Server Configuration

File: `src/api/server.py`

Configuration Setup

Update the startup event with your chosen services:

```
@app.on_event("startup")
async def startup_event():
    global pipeline

    # Configure your chosen services
    stt_config = {
        "api_key": os.getenv("DEEPGRAM_API_KEY"), # or your chosen STT
        "model": "nova-2"
    }

    llm_config = {
        "api_key": os.getenv("OPENAI_API_KEY"), # or your chosen LLM
        "model": "gpt-3.5-turbo",
        "temperature": 0.7
    }

    tts_config = {
        "api_key": os.getenv("ELEVENLABS_API_KEY"), # or your chosen TTS
        "voice_id": "21m00Tcm4TlvDq8ikWAM"
    }

    # Uncomment to initialize pipeline
    pipeline = await create_pipeline(stt_config, llm_config, tts_config)
```

Testing Your Implementation

1. Test Individual Components

Test Knowledge Base:

```
python src/utils/kb_test.py
```

Test STT:

```
curl -X POST http://localhost:8000/debug/stt -F "audio=@test.wav"
```


Test TTS:

```
curl "http://localhost:8000/chat/audio/Hello" --output test.mp3
```

2. Test Complete Pipeline

Start the server:

```
python -m src.api.server
```

Start the UI:

```
streamlit run streamlit_app.py
```

Use the Streamlit UI to test:

- Text chat (tests LLM + RAG)
- Audio chat (tests full STT → LLM → TTS pipeline)
- Health monitoring (shows component status)

3. API Testing

Text endpoint:

```
curl -X POST http://localhost:8000/chat/text \
-H "Content-Type: application/json" \
-d '{"text": "What is your return policy?"}'
```

Audio endpoint:

```
curl -X POST http://localhost:8000/chat/audio \
-F "audio=@question.wav" --output response.mp3
```

Service Selection Guide

Quick Start Combinations

1. Completely Free (No API keys needed):

- STT: OpenAI Whisper (local)
- LLM: Ollama with local model
- TTS: Edge TTS (Microsoft voices)

2. Mixed (Some free tiers):

- STT: OpenAI Whisper (local)
- LLM: OpenAI API (pay-per-use)
- TTS: Edge TTS (free)

3. Full API (Best quality):

- STT: Deepgram (free \$200 credits)
- LLM: OpenAI API (pay-per-use)
- TTS: ElevenLabs (free 10k chars/month)

Environment Variables

Set up your `.env` file based on chosen services:

```
# Copy and edit
cp .env.example .env
```

```
# Add your API keys for chosen services
STT_API_KEY=your_stt_key_here
LLM_API_KEY=your_llm_key_here
TTS_API_KEY=your_tts_key_here
```

Evaluation Criteria

Your implementation will be evaluated on:

Functionality (80%) - PRIMARY FOCUS

- **RAG Implementation** (25%): Does it retrieve relevant customer support information?
- **STT Quality** (20%): Accurate transcription of speech to text
- **TTS Quality** (20%): Natural-sounding speech synthesis

- **Pipeline Integration** (15%): Complete audio-in, audio-out workflow works end-to-end

Code Quality (10%)

- **Clean Implementation**: Well-structured, readable code
- **Error Handling**: Proper exception handling and logging
- **Async Patterns**: Correct use of `async/await`

Testing (10%)

- **Component Testing**: Individual components work correctly
- **Integration Testing**: Complete pipeline functions end-to-end
- **Edge Cases**: Basic error handling (empty inputs, API failures)

Note: Functionality is the primary focus. If core components don't work, code quality and tests won't compensate. Focus on getting your implementation working first, then improve code quality.

Common Issues & Solutions

STT Issues

- **Empty transcription**: Check audio format (WAV recommended) and ensure speech is present
- **API errors**: Verify API key and check service quotas
- **Format errors**: Convert audio to supported format (16kHz WAV works best)

LLM Issues

- **RAG not working**: Run `python src/utils/kb_test.py` to verify knowledge base setup
- **API quota**: Check your API usage and billing
- **Slow responses**: Consider using faster models (gpt-3.5-turbo vs gpt-4)

TTS Issues

- **Voice not found**: Use `get_available_voices()` method to see valid options

- **Quota exceeded:** Check monthly limits for free tiers
- **Audio quality:** Experiment with different models/voices

Pipeline Issues

- **Component not ready:** Check `/health` endpoint to see which components failed
- **Memory issues:** Large audio files may need chunking
- **Import errors:** Ensure all required packages are installed

Tips for Success

Implementation Tips

1. **Start with RAG:** It's the most critical component and easiest to test
2. **Test incrementally:** Don't try to implement everything at once
3. **Use the provided utilities:** `kb_test.py` and Streamlit UI are your friends
4. **Check the health endpoint:** Shows you exactly what's working/broken

Debugging Tips

1. **Enable verbose logging:** Check server logs for detailed error messages
2. **Use debug endpoints:** Test STT and TTS components individually
3. **Start simple:** Use shorter audio files and simple queries initially
4. **Check API documentation:** Each service has specific requirements

Performance Tips

1. **Choose appropriate models:** Balance quality vs speed vs cost
2. **Cache when possible:** Some models benefit from keeping connections open
3. **Handle timeouts:** Add appropriate timeout values for API calls
4. **Monitor usage:** Keep track of API quotas and costs

Bonus Challenges

Once you complete the core implementation, try these enhancements:

Advanced Features

- **Streaming Audio:** Implement real-time audio streaming for lower latency
- **Multi-language Support:** Handle multiple languages in the pipeline
- **Conversation Memory:** Add conversation history and context awareness
- **Voice Activity Detection:** Improve audio processing with VAD

Technical Improvements

- **Better Error Recovery:** Graceful fallbacks when components fail
- **Performance Optimization:** Reduce latency through caching and optimization
- **Custom RAG:** Enhance the knowledge base with more sophisticated retrieval
- **Audio Preprocessing:** Add noise reduction and audio enhancement

User Experience

- **Web Interface:** Create a better web UI for demonstration
- **Mobile Support:** Make the interface work on mobile devices
- **Voice Personas:** Different voice characteristics for different query types
- **Analytics:** Track usage patterns and common queries

Additional Resources

- **LangChain Documentation:** <https://python.langchain.com/>
- **ChromaDB Documentation:** <https://docs.trychroma.com/>
- **FastAPI Documentation:** <https://fastapi.tiangolo.com/>
- **Streamlit Documentation:** <https://docs.streamlit.io/>

Final Checklist

Before submission, ensure:

- ☐ RAG search returns relevant customer support information
- ☐ STT accurately transcribes speech to text
- ☐ TTS generates clear, natural-sounding speech
- ☐ Complete pipeline processes audio end-to-end
- ☐ Health endpoint shows all components as ready

- ☐ Streamlit UI works for both text and audio chat
- ☐ Error handling gracefully manages failures
- ☐ Code is clean, documented, and follows best practices

Good luck with your implementation! Remember to test each component thoroughly before moving to the next phase.